
Welcome Offer Access to everything. Now 30% off. [Upgrade now](#)

Terraform + lac + Coding ✓ VMware Workstation + Infrastructure +

Terraform + VMware Workstation 17 Pro: A Survival Guide



Mouhssineelh

Follow

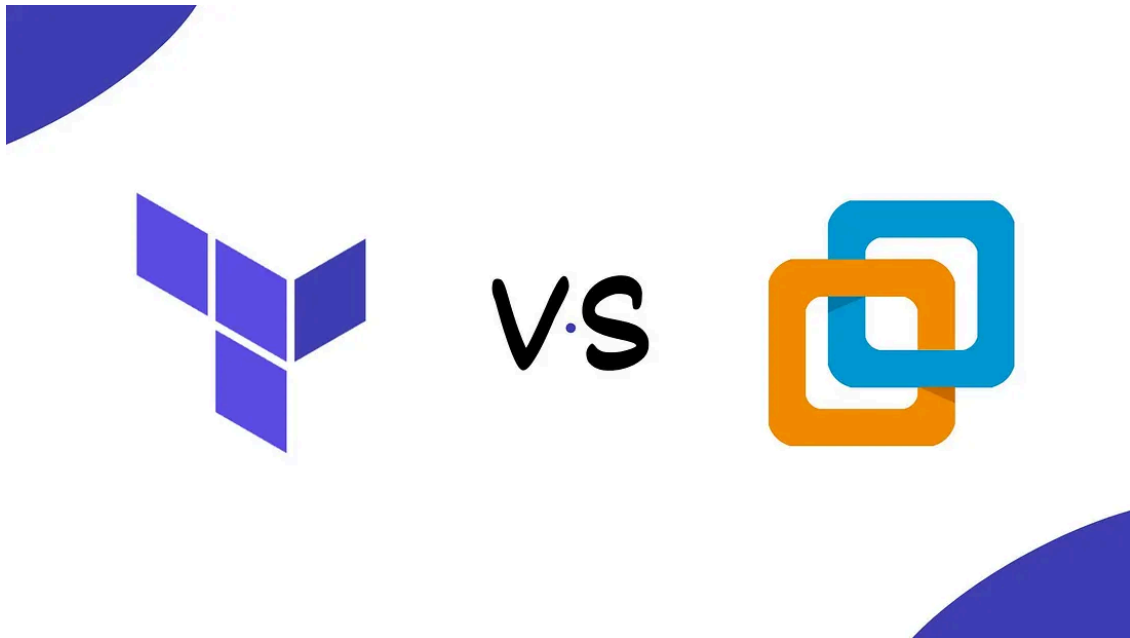
7 min read · May 6, 2026



54



2



The Struggle

This is a battle between a human, a tool, some broken packages, and elsudano himself.

After weeks of trying to manage infrastructure using Terraform, I finally managed to create a working script that can create and destroy VMs on VMware Workstation 17 Pro. During the research phase, I discovered that

Terraform is much better integrated with **Type 1 hypervisors** (like VMware ESXi or Proxmox): software that runs directly on the kernel.

Using those would have required wiping my entire laptop, which wasn't an option. So I set out to make it work with **VMware Workstation**, which is a **Type 2 hypervisor**: software that runs on top of an existing OS like Windows.

The only Terraform provider for VMware Workstation is

`elsudano/vmworkstation`. It's a community-maintained provider, not an official VMware or HashiCorp product, and it shows.

The newer versions (2.0.1) have a critical bug on Windows where the provider crashes with a Go panic (index out of range [0] with length 0) immediately after trying to create a VM, this is a confirmed bug reported in the provider's GitHub issues and has no fix at the time of writing. Version 1.0.4 is more stable, it creates the VM successfully but has a quirk where the path needs to match exactly where VMware Workstation decides to store the VM, not necessarily where you tell it to.

```
github.com/hashicorp/terraform-plugin-go@v0.28.0/tfprotov6/tfserver/server.go:941 +0x309
github.com/hashicorp/terraform-plugin-go/tfprotov6/internal/tfplugin6._Provider_ApplyResourceChange_Handler({0x14b4c80, 0xc0001ba460}, {0x161c2d8, 0xc00040a1e0},
0xc000442000, 0x0)
github.com/hashicorp/terraform-plugin-go@v0.28.0/tfprotov6/internal/tfplugin6/tfplugin6_grpc.pb.go:687 +0x1a6
google.golang.org/grpc.(*Server).processUnaryRPC(0xc0001f8200, {0x161c2d8, 0xc00040a060}, 0xc0001081e0, 0xc0002e0ea0, 0x1bfa1a8, 0x0)
google.golang.org/grpc@v1.73.0/server.go:1405 +0x1036
google.golang.org/grpc.(*Server).handleStream(0xc0001f8200, {0x161ccd0, 0xc0003fc000}, 0xc0001081e0)
google.golang.org/grpc@v1.73.0/server.go:1815 +0xb88
google.golang.org/grpc.(*Server).serveStreams.func2.1()
google.golang.org/grpc@v1.73.0/server.go:1035 +0x7f
created by google.golang.org/grpc.(*Server).serveStreams.func2 in goroutine 36
google.golang.org/grpc@v1.73.0/server.go:1046 +0x11d

Error: The terraform-provider-vmworkstation_v2.0.1.exe plugin crashed!

This is always indicative of a bug within the plugin. It would be immensely
helpful if you could report the crash with the plugin's maintainers so that it
can be fixed. The output above should help diagnose the issue.
```

Prerequisites

Before running anything, you need:

1. VMware Workstation 17 Pro installed on Windows
2. A **base VM** (the gold image that Terraform will clone), i used this one:<https://portal.cloud.hashicorp.com/vagrant/discover/bento/ubuntu-22.04>

3. vmrest running on Windows with credentials configured

Step 1: Prepare the Base VM

Terraform can't pull images from the web with this provider, unlike VirtualBox's `terra-farm` provider which supports direct image URLs. Instead, `vmrest` works by **cloning an existing VM**, so you need a pre-built base image registered in Workstation.

Download a ready-made VMware box from the Vagrant catalog:

- Go to <https://app.vagrantup.com/bento/boxes/ubuntu-22.04>
- Select the latest version → provider: `vmware_desktop` → architecture: `amd64`
- Download the `.box` file

Extract it: a `.box` file is just a renamed `.tar.gz`:

```
Rename-Item "installed-file-name.box" "bento-ubuntu-22.04.tar.gz"
tar -xzf bento-ubuntu-22.04.tar.gz #or use 7-zip on windows ila bghito
```

Open the extracted `.vmx` file in VMware Workstation (File → Open), boot it once, then power it off. It's now registered as your base VM. **It must always be powered off when Terraform runs:** `vmrest` cannot clone a running VM.

Important: Make sure you download the `amd64` variant, not `aarch64`. The ARM version will silently fail to boot on an Intel/AMD machine.

and also note that your username and password are both `vagrant` and that the keyboard is `qwerty` so you need to change that once and for all if you want.

Step 2: Start and Configure vmrest

vmrest is VMware's built-in REST API server. It's what Terraform talks to instead of controlling Workstation directly. You'll find it at:

```
C:\Program Files (x86)\VMware\VMware Workstation\vmrest.exe
```

Open PowerShell and configure credentials (only needed once):

```
cd "C:\Program Files (x86)\VMware\VMware Workstation"  
.\vmrest.exe --config #or .\vmrest.exe -C
```

This prompts you to set a username and password. After that, every time you want to use Terraform, just start vmrest and leave the terminal open:

```
.\vmrest.exe
```

vmrest will be available at `http://localhost:8697`. You can verify it's working by opening a browser and going to `http://localhost:8697/api/vms`, it will show you a message like invalid credentials which is fine, it shows that it's working and need only the correct credentials.

To get your base VM's ID (you'll need this for the Terraform script):

```
$cred = [Convert]::ToBase64String([Text.Encoding]::ASCII.GetBytes("username:pass  
[System.Text.Encoding]::UTF8.GetString((Invoke-WebRequest -Uri "<http://localhos
```

Step 3 (optional for who want to run this from an other VM): Expose vmrest to the VM Network

By default, vmrest only listens on 127.0.0.1 (localhost), meaning only the Windows machine itself can reach it. Since we're running Terraform from inside an Ubuntu VM, we need to make vmrest reachable from the VM's network.

The solution is a **Windows port proxy**, it listens on all network interfaces and forwards traffic to vmrest on localhost. Run this in an **admin PowerShell**:

```
netsh interface portproxy add v4tov4 listenport=8697 listenaddress=0.0.0.0 conne
```

What this does: any machine that can reach your Windows host on port 8697 will have their traffic forwarded to vmrest at 127.0.0.1:8697. Think of it as a transparent tunnel between the outside world and your local vmrest.



Also add a firewall rule to allow the traffic through (admin PowerShell):

```
New-NetFirewallRule -DisplayName "vmrest API" -Direction Inbound -Protocol TCP -
```

Note: The portproxy must be started **after** vmrest is already running, because both compete for port 8697. If you stop vmrest and try to restart it

while the portproxy is still active, vmrest will fail with a “port already in use” error. Always delete the portproxy first, restart vmrest, then re-add the portproxy:

```
# Delete portproxy
netsh interface portproxy delete v4tov4 listenport=8697 listenaddress=0.0.0.0
# Start vmrest, then re-add:
netsh interface portproxy add v4tov4 listenport=8697 listenaddress=0.0.0.0 connec
```

Step 4 (optional for who want to run this from an other VM): Set Up the Terraform Runner VM

Clone your base VM manually in Workstation (right-click → Manage → Clone → Full Clone) and name it `terraform-runner`. This is the VM you'll use to run Terraform, it stays separate from the base image.

Boot it, log in with `vagrant / vagrant`, then install Terraform:

```
wget -O- <https://apt.releases.hashicorp.com/gpg> | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] <https://apt.releases.hashicorp.com> ." | sudo tee /etc/apt/sources.list.d/hashicorp.list
sudo apt update && sudo apt install terraform git -y
```

Find the Windows host IP (the gateway the VM uses to reach Windows):

```
ipconfig #then find your pc's ip adress
```

Test that vmrest is reachable:

```
curl http://<windows-host-ip>:8697/api/vms
```

You should get `{"Code":1,"Message":"Authentication failed"}`, that means the connection works.

Step 5: The Terraform Script

Clone your repo and navigate to it:

```
git clone <https://github.com/your-username/your-repo.git>
cd your-repo
```

Here's the full script with explanation of each block below:

```
terraform {
  required_providers {
    vmworkstation = {
      source = "elsudano/vmworkstation"
      version = "1.0.4"
    }
    random = {
      source = "hashicorp/random"
      version = "~> 3.0"
    }
  }
  required_version = ">= 1.0.0"
}

provider "vmworkstation" {
  url      = "<http://host-ip-address:8697/api>"
  user     = "your-username"
  password = "your-password"
  https    = false
  debug    = false
}

resource "random_id" "vm_suffix" {
  byte_length = 4
}

resource "vmworkstation_vm" "my_vm" {
  sourceid = "Base-VM-ID"
```

Highlight Respond Share Private note

```

denomination = "my-terraform-vm-${random_id.vm_suffix.hex}"
description   = "VM created by Terraform"
path          = "C:\\Default-VMWare-Path\\my-terraform-vm-${random_id.vm_suffix.hex}"
processors    = 2
memory        = 2048
}
output "vm_name" {
  value = vmworkstation_vm.my_vm.denomination
}
output "vm_id" {
  value = vmworkstation_vm.my_vm.id
}

```

Block by Block Explanation

terraform {} block: The initialization block. Declares which providers Terraform needs to download before it can do anything. Think of providers as drivers, without the right driver, Terraform has no idea how to talk to VMware.

provider "vmworkstation" {} block: Configures the connection to vmrest. The `url` is the Windows host IP as seen from inside the Ubuntu VM. This will be different on every machine, always verify it with `ip route | grep default` from inside the runner VM.

resource "random_id" "vm_suffix" {} block: Generates a random 8-character hex string (like `a3f2c1b4`) each time Terraform runs. This is a workaround for a provider quirk: if a VM with the same name or path already exists in Workstation's inventory (even as a ghost from a previous failed run), Terraform errors out. By making every name unique, we sidestep the conflict entirely.

resource "vmworkstation_vm" "my_vm" {} block: The actual VM definition. `sourceid` is the ID of your base VM, the one Terraform will clone. It must be powered off. `denomination` is the display name in Workstation. `path` is where the `.vmx` file will be stored on the Windows host, it must match exactly where VMware Workstation's default VM location is set (check Workstation → Edit → Preferences → Workspace).

output blocks: Print the VM's name and ID after a successful apply, so you always know what was created.

Running It

```
terraform init      # download providers
terraform plan      # preview what will be created
terraform apply     # create the VM
terraform destroy   # destroy the VM
```

The `-parallelism=1` flag is optional as vmrest doesn't support concurrent API calls, so parallel execution will cause errors.

Things to Update on a New Machine

When running this on a different computer, update these three values in `main.tf`:

Open in app ↗



Medium



Search



Write



Workstation → Edit → Preferences → Workspace for default VM location

Known Limitations and Improvement Points

- **No image from URL** : unlike VirtualBox's provider, you always need a pre-existing base VM. If sharing this with classmates, they each need to manually set up their base image first.

- **Credentials in plain text** : the `user` and `password` are hardcoded in the script. A better approach would be to use environment variables (`VMWS_USERNAME`, `VMWS_PASSWORD`) or a `terraform.tfvars` file that's excluded from Git via `.gitignore`.
- **Path is machine-specific** : the `path` field points to a Windows path that's specific to your username and setup. Consider using a Terraform variable for this.
- **terraform destroy doesn't fully clean up** : after destroying, `vmrest` sometimes still has a ghost entry for the VM. Restarting `vmrest` clears it.
- **The portproxy doesn't survive reboots** : you need to re-add it every time Windows restarts. You could add it as a startup script to automate this.

References

[VmWare Workstation Pro Provider Documentation](#)

Terraform +

lac +

Coding ✓

Vmware Workstation +

Infrastructure +



Written by Mouhssineelh

5 followers · 6 following

Follow

